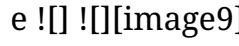
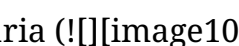


Administração e Otimização de Bancos de Dados Vetoriais para Aplicações de Inteligência Artificial usando PostgreSQL (pgvector)

Fundamentação Teórica dos Bancos Vetoriais

No âmbito do aprendizado de máquina contemporâneo, a representação de dados não estruturados fundamenta-se no mapeamento de conceitos semânticos em espaços vetoriais contínuos de alta dimensionalidade¹. Matematicamente, um *embedding* é uma função de projeção , onde um objeto arbitrário  (como um texto, uma imagem ou um áudio) é codificado em um vetor de números reais com dimensionalidade  tipicamente compreendida entre  e ¹. Geometricamente, o vetor resultante  representa coordenadas em um espaço de Hilbert de alta dimensão, onde a proximidade geométrica entre dois pontos reflete a similaridade semântica dos objetos originais³. O principal desafio operacional em bancos de dados vetoriais reside em calcular a proximidade dessas coordenadas com eficiência escalar. O pgvector implementa operadores especializados baseados em três métricas de distância fundamentais para determinar a similaridade entre vetores⁵.

Distância L2 (Euclidiana)

A Distância L2, ou Euclidiana, calcula o comprimento do segmento de reta que une dois pontos no espaço vetorial³. Matematicamente, é expressa pela raiz quadrada da soma das diferenças quadradas entre as coordenadas correspondentes de dois vetores $\mathbf{u}$ e $\mathbf{v}$ . Este operador é representado no pgvector pelo caractere `<->`⁵. Sob a ótica de arquiteturas de Inteligência Artificial, a Distância L2 é altamente sensível à magnitude absoluta dos vetores⁷. Consequentemente, o seu caso de uso ideal envolve dados espaciais, leituras de sensores físicos ou representações numéricas onde a escala ou frequência de ocorrência de um atributo influencie diretamente o seu peso semântico⁷. Se os *embeddings* de texto forem previamente normalizados para possuírem norma unitária ($\|\mathbf{u}\| = 1$) , a ordenação obtida pela Distância L2 torna-se equivalente à similaridade de cosseno, embora a distância de cosseno seja preferida para preservar a portabilidade conceitual⁷.

Produto Escalar (Inner Product)

O Produto Escalar mede a projeção de um vetor sobre o outro, ponderada pelas magnitudes de ambos³. A sua expressão matemática é dada pelo somatório do produto das componentes correspondentes:

$$\mathbf{u} \cdot \mathbf{v} = \sum u_i v_i$$

No pgvector, o operador associado é o `<#>`, o qual retorna o produto escalar negativo ($-\mathbf{u} \cdot \mathbf{v}$) ³. Esta negação é uma imposição arquitetônica do otimizador do PostgreSQL: como o SGBD realiza buscas ordenadas de forma ascendente (menor valor primeiro), a negação garante que os vetores com maior produto escalar (maior similaridade) sejam retornados no topo da consulta⁷.

O Produto Escalar é o mecanismo de busca padrão para algoritmos de Busca pelo Máximo Produto Escalar (MIPS - *Maximum Inner Product Search*)⁷. É o modelo matemático ideal para sistemas de recomendação onde os vetores representam

interações usuário-item não normalizadas⁷. Nesses cenários, a magnitude do vetor indica o nível de engajamento ou a força do sinal de confiança, enquanto a direção indica a afinidade de preferências⁹.

Similaridade de Cosseno

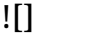
A Similaridade de Cosseno avalia exclusivamente o alinhamento angular entre dois vetores, neutralizando quaisquer disparidades introduzidas por suas magnitudes⁷.

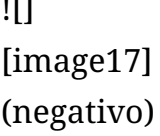
Geometricamente, calcula o cosseno do ângulo  formado entre as direções dos vetores no espaço multidimensional⁷. O pgvector calcula a Distância de Cosseno, definida como o complemento da similaridade de cosseno em relação à unidade⁸:



Representada pelo operador `<=>`, a distância de cosseno assume valores estritamente no intervalo ⁷. É a métrica mais amplamente adotada em aplicações de processamento de linguagem natural (NLP) e Grandes Modelos de Linguagem (LLMs)⁷.

Em modelos como os da OpenAI, Cohere ou SentenceTransformers, a semântica de um texto está contida na orientação de seu vetor representativo, e não na frequência absoluta das palavras, que inflaria artificialmente a magnitude do vetor de documentos mais longos⁸. Ao normalizar o cálculo pelo produto das normas, a similaridade de cosseno garante que parágrafos de tamanhos discrepantes com o mesmo teor semântico apresentem distância próxima de zero⁷.

Métrica de Distância	Operador SQL	Intervalo de Saída	Sensibilidade à Magnitude	Caso de Uso Primário em IA
L2 (Euclidiana)	<code><=></code>	 	Alta	Dados de sensores, coordenadas físicas ⁷ .

Métrica de Distância	Operador SQL	Intervalo de Saída	Sensibilidade à Magnitude	Caso de Uso Primário em IA
Produto Escalar	$\langle \# \rangle$	 (negativo)	Total	Sistemas de Recomendação (MIPS), score de afinidade7.
Distância de Cosseno	$\langle = \rangle$	 (Nula)	Nula (apenas ângulo)	Embeddings de texto de LLMs (OpenAI, Cohere)7.

A Maldição da Dimensionalidade e a Intrinsecabilidade do Espaço Vetorial

Ao administrar sistemas de busca vetorial, o arquiteto de dados confronta-se com a maldição da dimensionalidade¹. Em espaços métricos de alta dimensão, a distância entre o ponto mais próximo e o mais distante converge para uma diferença marginal, tornando o particionamento espacial tradicional ineficiente. Pesquisas empíricas revelam que a revocação de algoritmos aproximados como o HNSW está diretamente atrelada à Dimensão Intrínseca Local (LID - *Local Intrinsic Dimensionality*) dos vetores¹⁰.

A LID mede a dimensionalidade real do subespaço onde os dados de fato se distribuem, a qual é frequentemente inferior à dimensão nominal do vetor¹⁰. Desvios estatísticos na sequência de inserção dos dados podem induzir anomalias na topologia do grafo HNSW, reduzindo a revocação em até 12,8 pontos percentuais quando os vetores são inseridos ordenados por sua assinatura de LID¹⁰. Assim, a distribuição estatística e a ordem de carga dos dados constituem fatores críticos para a estabilidade da infraestrutura de busca¹⁰.

Arquitetura do SGBD: PostgreSQL + pgvector

Justificativa Técnica da Escolha Arquitetural

A consolidação de infraestruturas de Inteligência Artificial motivou o surgimento de bancos de dados vetoriais dedicados e proprietários, como Pinecone, Milvus ou Qdrant¹¹. Contudo, a adoção do PostgreSQL estendido com o pgvector fundamenta-se em princípios rigorosos de engenharia de software e teoria de bancos de dados, oferecendo vantagens estratégicas incontornáveis¹.

Critério de Comparação	PostgreSQL + pgvector	SGBDs Vetoriais Dedicados
Consistência de Dados	Transações ACID nativas, MVCC estrito ¹³ .	Consistência eventual ou sincronização bidirecional complexa ¹ .
Complexidade de Infraestrutura	Instância única, menor sobrecarga operacional ¹ .	Múltiplos serviços, pipelines de sincronização ETL adicionais ⁹ .
Flexibilidade de Consulta	Junções SQL nativas, CTEs, indexação híbrida ⁹ .	Filtragem restrita por metadados, APIs proprietárias DSL ¹⁵ .
Segurança e Conformidade	Segurança em nível de linha (RLS), auditorias consolidadas ¹⁸ .	Políticas de acesso restritas ou replicadas na aplicação ¹² .

Critério de Comparação	PostgreSQL + pgvector	SGBDs Vetoriais Dedicados
Recuperação de Desastres	Backup consistente, replicação física e PITR ¹³ .	Mecanismos proprietários de snapshot em cloud ¹⁵ .

A fragmentação arquitetônica inerente ao uso de um banco de dados relacional clássico em conjunto com um motor vetorial especializado introduz riscos de sincronização¹¹. Quando uma entidade de negócio é atualizada, a modificação correspondente no banco vetorial ocorre de forma assíncrona, criando janelas de inconsistência em que dados desatualizados são expostos a pipelines de RAG¹¹. No PostgreSQL, as atualizações relacionais e a persistência de novos vetores de embedding ocorrem sob a égide da mesma transação ACID, eliminando falhas de gravação parcial¹³.

Gerenciamento de Armazenamento Físico de Dados Vetoriais

O PostgreSQL opera nativamente com um modelo de armazenamento físico baseado em páginas de tamanho fixo de 8 KB¹⁴. Toda tupla gravada em uma tabela deve ser estruturada de forma a respeitar esse limite ou delegar o excedente para mecanismos de transbordo¹⁴.

O tipo de dado vector(n) fornecido pelo pgvector é implementado em nível de código C como um tipo de dado de tamanho variável conhecido internamente como varlena². A representação física de uma estrutura vetorial em disco é descrita pela seguinte declaração de estrutura C na base de código da extensão²:

```
C
typedef struct Vector
{
    int32 vl_len_; /* Header padrão varlena do PostgreSQL (contém
```

```

o tamanho do bloco) */
int16 dim; /* Quantidade de dimensões ativas do vetor */
int16 unused; /* Padding para alinhamento de memória de 8
bytes */
float x[FLEXIBLE_ARRAY_MEMBER]; /* Array contínuo de
valores de ponto flutuante de 32 bits (float32) */
} Vector;

```

A partir desse leiaute físico, o espaço em bytes consumido exclusivamente pelos dados do vetor em disco é calculado como:

![][image18]

Onde ![][image3] é a dimensão do vetor e ![][image19]

representa o overhead fixo do cabeçalho de metadados². Um vetor padrão de 1536 dimensões (comum em modelos como o text-embedding-3-small da OpenAI) consome exatamente ![][image20] bytes de armazenamento bruto de payload²².

Quando uma linha contendo um vetor desse tamanho é inserida, ela excede o limite padrão de ativação do mecanismo TOAST (*The Oversized-Attribute Storage Technique*), que é de 2 KB²⁵. O TOAST intercepta atributos de tamanho excessivo para evitar que uma única linha ocupe toda a página de 8 KB do PostgreSQL, o que penalizaria gravemente as operações de leitura sequencial na tabela principal¹⁴.

Até a versão 0.5.x do pgvector, a estratégia padrão de armazenamento do TOAST para vetores era EXTENDED, que permitia a compressão inline do array de floats antes de armazená-lo externamente²⁵. Contudo, vetores de alta dimensionalidade gerados por IA contêm valores de ponto flutuante pseudoaleatórios altamente entrópicos, cuja taxa de compressão real aproxima-se de zero²⁵. O processo de compressão consumia valiosos ciclos de CPU do servidor sem gerar economia física de disco²⁵.

A partir do pgvector 0.6.0, o tipo de dados foi reconfigurado para utilizar a estratégia EXTERNAL por padrão²⁵. Sob esta estratégia, o PostgreSQL grava o vetor de forma direta e sem compressão nas páginas da tabela TOAST secundária, mantendo na tabela principal apenas um ponteiro físico de 18 bytes¹⁴. Isso otimiza os escaneamentos rápidos da tabela principal, embora

requiera acessos indiretos adicionais às páginas do TOAST quando o vetor precisa ser recuperado na cláusula de retorno da consulta²⁶.

Embora o mecanismo TOAST solucione com sucesso a persistência de vetores de tamanho superior ao limite de página do PostgreSQL, ele introduz uma limitação crítica nos algoritmos de indexação²¹. O PostgreSQL não permite construir índices diretos sobre linhas cujos atributos estejam fragmentados ou armazenados fisicamente fora do bloco da tabela principal por meio do TOAST²¹. Por essa razão, os índices HNSW e IVFFlat do pgvector impõem limites estruturais rígidos de dimensionalidade:

- O tipo padrão vector (baseado em floats de 32 bits) está limitado a no máximo 2.000 dimensões para fins de indexação⁶.
- Para suportar modelos de alta dimensionalidade como o text-embedding-3-large (3.072 dimensões), o administrador deve utilizar o tipo de precisão reduzida halfvec (floats de 16 bits), que eleva o limite de indexabilidade para até 4.000 dimensões e reduz o consumo de armazenamento pela metade⁶.
- Outras alternativas incluem o tipo bit para quantização binária (suportando até 64.000 dimensões) e sparsevec para dados esparsos com até 1.000 elementos não nulos².

O Conceito de Busca Híbrida (Hybrid Search)

A essência operacional do pgvector consiste na capacidade de unificar pesquisas estruturadas relacionais e buscas aproximadas por similaridade vetorial em uma única expressão SQL compilável⁹. No entanto, o otimizador de consultas do PostgreSQL precisa balancear o custo de execução de filtros de atributos contra o custo de travessia do grafo vetorial²⁸.

O planejador de consultas estima a seletividade dos filtros relacionais com base nas estatísticas geradas pelo processo ANALYZE⁶. Duas estratégias principais de execução podem ocorrer:

Planejamento de Filtragem via Índices Relacionais (Pre-filtering)

Se o filtro relacional (por exemplo, `WHERE tenant_id \= '...'`) apresentar alta seletividade, retornando um número reduzido de linhas, o planejador opta por realizar um escaneamento de índice tradicional (B-Tree ou Hash)²⁸. As tuplas resultantes são filtradas em memória e o cálculo de distância vetorial é computado de forma exata (K-NN linear) apenas sobre este subconjunto qualificado, ignorando completamente o índice vetorial²¹. Esta abordagem garante  de revocação e baixo tempo de resposta sob alta seletividade²¹.

Planejamento de Escaneamento Iterativo do Índice (Iterative Index Scan)

Quando os filtros relacionais possuem baixa seletividade, ou seja, qualificam uma parcela substancial da tabela, o custo de avaliar linearmente os vetores torna-se proibitivo²⁸. O PostgreSQL decide utilizar o índice vetorial (HNSW ou IVFFlat) para guiar a recuperação⁹.

Contudo, se o planejador aplicar um filtro estático após extrair os candidatos aproximados do índice (pós-filtragem), a query pode sofrer do problema de superfiltragem (*overfiltering*)²⁸: se o índice extrair apenas os 40 candidatos mais próximos de forma absoluta (`ef_search \= 40`) e a maioria deles falhar no filtro relacional, o resultado final conterá menos registros do que o limite solicitado pela cláusula `LIMIT`³¹.

Para solucionar este dilema, o pgvector 0.8.0 introduziu a funcionalidade de escaneamento iterativo de índice¹. O processo opera em ciclos contínuos de execução integrada em nível de motor³⁴:

O planejador inicia uma busca no grafo HNSW rastreando um número limitado de nós determinados pelo buffer de busca `ef_search`³⁴. À medida que os candidatos são ejetados pelo índice em ordem aproximada de distância, o PostgreSQL aplica os filtros relacionais inline sobre seus metadados³⁴.

Caso o número de tuplas validadas que passaram pelos filtros

não satisfaça o limite exigido pela cláusula LIMIT, o motor do banco de dados não interrompe a execução: ele retoma a travessia de onde parou, varrendo de forma incremental novos caminhos do grafo até obter o número necessário de correspondências ou atingir o limite físico estipulado pela variável `hnsw.max_scan_tuples`³¹. Isso equilibra a precisão semântica e a consistência relacional em uma única query otimizada³⁴.

A sintonia do escaneamento iterativo é configurada pelo parâmetro de configuração global (GUC) `hnsw.iterative_scan`, que aceita três modos de comportamento operacional³¹:

- `off`: Desativa o comportamento iterativo, reproduzindo a pós-filtragem estática das versões anteriores (maior velocidade, alto risco de superfiltragem)³¹.
- `strict_order`: Garante a preservação estrita da ordenação por distância exata ao longo de todas as iterações de busca, de modo que nenhum vetor mais próximo seja preterido, à custa de maior latência de execução³¹.
- `relaxed_order`: Permite pequenas variações aproximadas na ordenação dos resultados intermediários retornados entre as iterações, otimizando de forma significativa a velocidade e o throughput das queries em ambientes de produção³¹.

Algoritmos de Indexação e Administração Física

Funcionamento Interno do IVFFlat e HNSW

Os algoritmos de indexação de aproximação (ANN) implementados no pgvector oferecem soluções distintas para o compromisso clássico entre velocidade de busca, tempo de compilação e custo de armazenamento³⁵.

O **IVFFlat (Inverted File Flat)** opera dividindo o espaço vetorial por meio do algoritmo de agrupamento *k-means* para formar células de Voronoi³⁵. O parâmetro de compilação `lists` define a

quantidade de agrupamentos (centroides) gerados³⁸. Durante a compilação do índice, o pgvector realiza passadas de treinamento sobre os dados existentes para fixar a posição geométrica de cada centroide²⁸. Cada vetor da tabela é então associado à lista invertida do centroide mais próximo³⁶. No momento da busca, o vetor de consulta é comparado apenas com os centroides mais próximos (quantidade definida pelo parâmetro dinâmico *probes*), restringindo o cálculo linear exato de distância exclusivamente às listas associadas a esses agrupamentos³³.

Por outro lado, o **HNSW (Hierarchical Navigable Small World)** baseia-se na estruturação de grafos de proximidade organizados em camadas hierárquicas⁴¹. O design estrutural inspira-se no comportamento probabilístico das listas de saltos (*skip lists*) para estender a busca de caminhos rápidos a redes de mundo pequeno (NSW) multidimensionais³¹.

A Camada 0 (base) abriga absolutamente todos os nós e vetores indexados, enquanto as camadas superiores hospedam subconjuntos exponencialmente mais esparsos de dados³¹. A altura máxima de inserção de um nó é determinada de forma probabilística na inserção pela equação de decaimento exponencial $![[image22]]$, onde o fator multiplicador ótimo é definido teoricamente por $![[image23]]$ ⁴¹.

Camada 2 (Esparsa - Saltos Largos)

Ponto de Entrada ---> Nó A -----> Nó F

| |

Camada 1 (Intermediária) v

Ponto de Entrada ---> Nó A -----> Nó C -----> Nó F

| | |

Camada 0 (Completa - Todos os Nós) v v

Ponto de Entrada ---> Nó A -> Nó B -> Nó C -> Nó D -> Nó E -> Nó F
(Alvo)

A velocidade do HNSW reside nesta estrutura multinível³¹. A busca inicia-se na camada mais alta por meio de uma busca gananciosa (*greedy search*), onde o algoritmo calcula a distância do vetor de consulta em relação aos vizinhos conectados ao

ponto de entrada ativo⁴¹.

Uma vez localizado o nó local mais próximo naquela camada, o algoritmo desce verticalmente para a camada inferior, utilizando este nó como novo ponto de partida para a busca gananciosa local³¹. O processo repete-se sucessivamente até atingir o nível basal (Camada 0), reduzindo a travessia a uma complexidade de tempo média de $O(\sqrt{n})$ ⁴⁵.

Em contrapartida, o IVFFlat apresenta complexidade de busca que depende fortemente da distribuição uniforme dos vetores nos clusters de Voronoi³³. Se houver desbalanceamento de clusters sem reindexação, a busca degrada para o comportamento linear em agrupamentos densos⁴⁷.

Conectividade do HNSW e Parâmetros Críticos de Construção

A qualidade topológica do grafo HNSW é governada diretamente pelos parâmetros configurados no momento de sua criação⁵:

- **m (Número máximo de conexões bidirecionais por elemento):** Define o limite máximo de arestas que cada nó pode estabelecer com vizinhos em cada camada do índice⁵. A camada base (Camada 0) constitui uma exceção estrutural fundamental: ela permite até $2m$ conexões por nó⁴³. Essa duplicação de limites na base é implementada para garantir a conectividade global do grafo e evitar que agrupamentos isolados de dados fiquem inacessíveis à travessia, mantendo o grafo resiliente a remoções e minimizando a existência de mínimos locais⁴³. O valor padrão no pgvector é $m=16$ (com variação típica entre $m=8$ e $m=32$)⁵.
- **ef_construction (Tamanho do buffer dinâmico de construção):** Controla o tamanho da fila de prioridades que rastreia os candidatos a vizinhos mais próximos durante a compilação do grafo⁵. Um valor elevado (padrão 100) força o algoritmo a inspecionar uma área maior do espaço métrico antes de decidir quais arestas fixar, otimizando a qualidade das pontes e a

acurácia futura de busca, à custa de um aumento linear no tempo total de construção do índice⁵.

O ajuste desses parâmetros exige a compreensão de um compromisso triplo entre tempo de compilação, consumo de memória RAM e taxa de revocação (*recall*) das consultas⁵.

Configuração de Parâmetros	Tempo de Build	Consumo de RAM	Taxa de Recall	QPS (Consultas/seg)
Padrão (m = 16, ef_c = 64) ³⁸	Baixo ³⁰	Mínimo ³⁰	Moderada (~95%)	Alta ⁴¹
Otimizado p/ Busca (m = 32, ef_c = 128) ³⁰	Moderado ³⁰	Médio (~20% mais arestas) ³³	Alta (~98%) ²³	Moderada
Alta Precisão (m = 64, ef_c = 256) ³⁵	Alto	Elevado (dobro do padrão) ³⁵	Altíssima (>99%)	Baixa

Sintonia Fina de `hnsw.ef_search` em Tempo de Execução

Diferente dos parâmetros de criação que são imutáveis após a compilação do índice, o parâmetro `hnsw.ef_search` (padrão `1`)^[image30] atua em tempo de execução para controlar a largura da busca gananciosa na Camada 06. Ele define o tamanho máximo da fila de prioridades que mantém os candidatos a vizinho mais próximo durante o rastreamento final⁸.

O ajuste dinâmico do `hnsw.ef_search` permite ao administrador calibrar o sistema sob demanda para diferentes fluxos de trabalho⁸:

- **Priorização de Vazão (QPS):** Configurar `hnsw.ef_search` para valores menores (ex: `1`^[image31] ou `1`^[image32])

reduz o número total de cálculos de distância por query⁵. Essa configuração é indicada para sistemas de alto tráfego concorrente, como preenchimento automático de busca, onde a latência de milissegundos únicos é mais importante do que encontrar todas as correspondências semânticas perfeitas³³.

- **Priorização de Revocação (Recall):** Elevar o parâmetro para valores entre 10^{-3} e 10^{-4} expande a varredura do grafo antes de interromper a busca⁵. É a configuração recomendada para sistemas de RAG de alta fidelidade (como diagnósticos jurídicos ou médicos), onde a perda de um único contexto semântico relevante pode comprometer a resposta gerada pelo LLM³⁵.

Administração Física e Ajuste de Performance (Tuning)

Dimensionamento Físico de Memória RAM para Índices HNSW

Diferente de índices tradicionais como B-Tree, o HNSW exige que a sua estrutura de nós e listas de adjacência resida integralmente na memória RAM do servidor para evitar a degradação de desempenho²⁸. Devido ao padrão aleatório de travessia do grafo, qualquer necessidade de paginação física para ler blocos de disco (*page faults*) eleva a latência da consulta por ordens de grandeza²⁸.

Para estimar com precisão o espaço ocupado pelo índice HNSW na memória buffers do PostgreSQL, o administrador deve calcular a pegada física média por vetor indexado utilizando a equação estrutural⁴⁸:

$$V \cdot (1 + \frac{L}{M})$$

Onde:

- V é o volume total de vetores indexados⁴⁸.
- M é a dimensão do vetor (ex: 1536)⁴⁸.

- m é o tamanho de representação de cada coordenada em precisão simples (float32)⁴⁸.
- k é o número de conexões máximas por nó³⁸.
- e é o multiplicador empírico para computar o overhead das conexões em múltiplas camadas do HNSW⁴⁸.
- p representa o tamanho de representação de cada ponteiro de vizinho em nível físico de página⁴⁸.

A tabela a seguir apresenta os requisitos de memória estimados sob diferentes dimensionalidades e volumes de dados para orientar o provisionamento de hardware, adicionando uma margem de segurança de 1.25 recomendada para acomodar o overhead de páginas físicas do PostgreSQL e o cache operacional²³.

Volume de Vetores (N)	Dimensão (d)	Conexões (m)	Tamanho Bruto do Índice	RAM Mínima Recomendada
100.000 (100k)	d (ex: BERT) ⁹	m [cite: 38]	m [image42] MB ⁴⁸	$1.25 \times m$ [image43] MB
100.000 (100k)	d (ex: OpenAI) ²²	m [image26] [cite: 38]	m [image45] MB ⁴⁸	$1.25 \times m$ [image46] MB
1.000.000 (1M)	d (ex: BERT) ⁹	m [image26] [cite: 38]	m [image47] GB	$1.25 \times m$ [image48] GB
1.000.000 (1M)	d (ex: OpenAI) ²²	m [image26] [cite: 38]	m [image49] GB ⁴⁸	$1.25 \times m$ [image50] GB ⁴⁸

Volume de Vetores (N)	Dimensão (d)	Conexões (m)	Tamanho Bruto do Índice	RAM Mínima Recomendada
1.000.000 (1M)	[image44] (ex: OpenAI) ²²	[image51] [cite: 48]	[image52] GB48	[image53] GB
10.000.000 (10M)	[image44] (ex: OpenAI) ²²	[image26] [cite: 38]	[image54] GB	[image55] GB

Parâmetros Recomendados para o postgresql.conf

A otimização de consultas vetoriais complexas exige o ajuste de parâmetros globais do PostgreSQL para assegurar a eficiência de cache do grafo e paralelizar tarefas de infraestrutura²⁰.

Ini, TOML

Configurações otimizadas para um servidor dedicado de 32 GB de RAM e 8 vCPUs

shared_buffers \= 12GB # Aloca 37.5% da RAM para reter o HNSW quente em cache [cite: 20, 54]

effective_cache_size \= 24GB # Indica a disponibilidade total de cache de arquivos (75% da RAM) [cite: 20]

work_mem \= 64MB # Evita escrita temporária em disco para ordenações por sessão [cite: 20, 54]

maintenance_work_mem \= 4GB # Aloca espaço para compilação rápida do grafo HNSW

max_parallel_maintenance_workers \= 4 # Habilita

trabalhadores paralelos na compilação do HNSW [cite: 23, 41, 55]

max_parallel_workers_per_gather \= 2 # Limita o paralelismo em buscas híbridas simultâneas [cite: 54, 56]

O parâmetro `maintenance_work_mem` desempenha papel crítico na velocidade de criação do índice⁶. Durante a execução do comando `CREATE INDEX ... USING hnsw`, o PostgreSQL aloca as estruturas dinâmicas de busca gananciosa diretamente neste segmento de memória⁴¹. Se o volume de dados exceder o limite configurado, o PostgreSQL emitirá a seguinte mensagem de alerta no log⁶:

```
NOTICE: hnsw graph no longer fits into
maintenance_work_mem after X tuples
DETAIL: Building will take significantly more time.
HINT: Increase maintenance_work_mem to speed up builds.
```

Este aviso indica que o grafo ultrapassou o espaço em memória reservado e que o motor do banco de dados iniciou a gravação de arquivos temporários em disco, o que causa degradação no tempo de construção do índice⁶. Para cargas acima de 1 milhão de vetores de 1536 dimensões, recomenda-se configurar `maintenance_work_mem` para no mínimo 4 GB durante a execução do `reindex`⁶.

Estratégias de Manutenção Física de Índices Vetoriais

À medida que novas linhas são inseridas, atualizadas ou excluídas do banco de dados, os índices vetoriais sofrem degradação estrutural silenciosa²⁸.

Diferente de tabelas puramente relacionais, onde a fragmentação e o espaço de linhas mortas decorrentes de atualizações (`UPDATE`) e deleções (`DELETE`) são contornados pelo processo padrão do `VACUUM`¹⁴, o índice HNSW não exclui imediatamente os caminhos físicos de arestas que conectam nós de elementos excluídos do banco de dados²⁸. Em vez disso, os nós correspondentes nas páginas de indexação recebem apenas marcadores de desativação lógica (conhecidos no jargão do `pgvector` como *tombstones*)²⁸.

O acúmulo excessivo desses nós mortos de vetores desativados cria vazios estruturais na topologia do grafo de travessia²⁸.

Quando a query de similaridade de cosseno caminha pelo grafo, ela passa a perder tempo avaliando arestas que levam a caminhos mortos, gerando o fenômeno de degradação de revocação (quando o índice deixa de encontrar vizinhos corretos) e inflacionando os tempos de p95/p99 de execução devido aos desvios de rota de busca²⁸.

Protocolo de Manutenção Recomendado para DBAs

- **Monitoramento de Desempenho e Coleta de Métricas:**

Estabelecer uma rotina de monitoramento estatístico periódico, medindo a taxa de acerto de cache de buffer de bloco por meio das tabelas de sistema do PostgreSQL (`pg_statio_user_indexes`)⁵². Se a taxa de acertos cair abaixo de , significa que o índice HNSW ultrapassou os limites saudáveis de RAM e está executando paginação física em disco⁵².

- **Teste de Integridade de Revocação (Recall Evaluation):**

Periodicamente, a aplicação deve rodar uma bateria controlada de queries vetoriais exatas (utilizando ORDER BY clássico sem o uso de índices HNSW) contra uma amostra de vetores novos e comparar os IDs resultantes com as respostas geradas sob a indexação HNSW⁴⁴. Caso a divergência de resultados (*recall loss*) ultrapasse a barreira de tolerância estipulada para a aplicação (geralmente abaixo de )⁵⁷), é o indicador definitivo de necessidade de intervenção estrutural de indexação¹⁶.

- **Reconstrução Incremental e Online:** A execução da reconstrução deve evitar o comando clássico `REINDEX INDEX index_name`, que bloqueia de forma exclusiva as transações de escrita na tabela ativa de RAG²⁸. Deve-se priorizar o comando concorrente:

SQL

```
REINDEX INDEX CONCURRENTLY  
idx_document_chunks_hnsw_cosine;
```

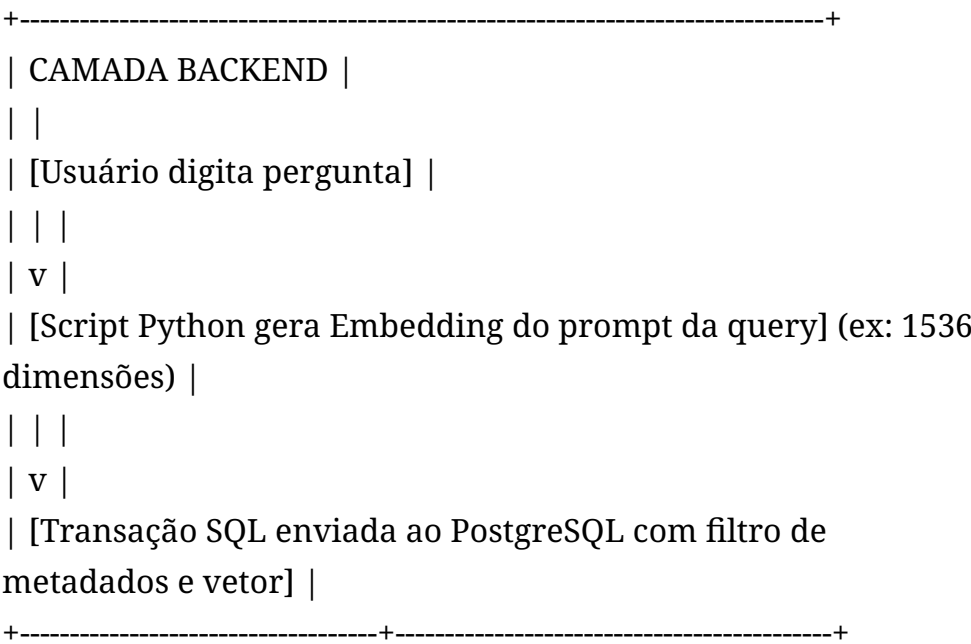
O PostgreSQL aloca processos paralelos em background para construir um grafo HNSW espelhado e completamente novo

com base nos dados consolidados mais recentes da tabela, sem suspender as queries simultâneas e escritas da aplicação de produção²⁸. Uma vez finalizado o processamento físico e validadas as conexões, as referências de catálogo são atualizadas atômicamente e o grafo antigo fragmentado é liberado do pool de páginas físicas em disco de forma transparente²⁸.

Proposta Prática e Arquitetura de Demonstração (RAG)

Para garantir o isolamento físico de dados sensíveis entre clientes de uma plataforma multi-inquilino (*multi-tenant*) e evitar o vazamento de informações contextuais em pipelines de RAG, projeta-se abaixo a arquitetura de banco de dados e backend¹².

A modelagem separa os documentos mestre de seus fragmentos vetoriais associados (*chunks*), permitindo atualizações incrementais e re-fragmentação sem perda de consistência⁵⁸. Adicionalmente, denormaliza-se o identificador de inquilino (*tenant_id*) diretamente na tabela de fragmentos para possibilitar filtragens em nível de memória RAM antes do cálculo exaustivo de similaridade vetorial⁵⁸.



```

|
v
+-----+-----+
| POSTGRESQL ENGINE |
| |
| 1. Filtro relacional seletivo rápido por B-Tree (ex: WHERE
tenant_id \= '...') |
| 2. Ativação do 'relaxed_order' iterative scan para recuperar K
candidatos |
| 3. Travessia rápida no Grafo HNSW em cache para buscar
correspondências |
| 4. Retorno ordenado por distância de cosseno aproximada |
+-----+-----+
|
v
+-----+-----+
| CAMADA BACKEND |
| |
| [Contextos enriquecidos extraídos do BD] |
| | |
| v |
| [Montagem do Prompt do LLM: Pergunta + Contextos
semânticos do Postgres] |
| | |
| v |
| [Envio do Prompt consolidado para API do LLM] (DeepSeek /
GPT-4) |
| | |
| v |
| [Geração da Resposta grounded final para o usuário] |
+-----+

```

Script de Implementação do Banco de Dados (SQL)

SQL

```

-- Ativação da extensão pgvector no banco de dados corrente
CREATE EXTENSION IF NOT EXISTS vector; --

```

-- Tabela principal para armazenamento dos Documentos

Mestre (Metadados Globais)

```
CREATE TABLE documents (  
id BIGSERIAL PRIMARY KEY,  
title TEXT NOT NULL,  
source_url TEXT,  
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Tabela especializada para retenção de fragmentos de texto

(Chunks) e embeddings associados

```
CREATE TABLE document_chunks (  
id BIGSERIAL PRIMARY KEY,  
document_id BIGINT REFERENCES documents(id) ON DELETE  
CASCADE, -- Garante integridade referencial [cite: 13]  
chunk_index INT NOT NULL,  
content TEXT NOT NULL,  
embedding vector(1536), -- Tipo nativo compatível com modelos  
OpenAI / Ada [cite: 2, 22]  
tenant_id UUID NOT NULL, -- UUID para isolamento lógico  
rigoroso multi-tenant  
metadata JSONB, -- Tags adicionais para busca híbrida  
estruturada
```

```
\-- Geração inline e dinâmica de coluna tsvector em português para busca full-t  
content\tsv tsvector GENERATED ALWAYS AS (to\tvector('portuguese', content))
```

```
\-- Restrição de unicidade para evitar inserções em duplicidade do mesmo fragme  
UNIQUE (document\_id, chunk\_index)
```

```
);
```

-- Indexação clássica relacional B-Tree para otimizar filtros em
nível de chave de inquilino

```
CREATE INDEX idx_document_chunks_tenant ON  
document_chunks (tenant_id); --
```

```
-- Criação do índice vetorial HNSW com otimização estrutural de
barreira de classe
-- Utiliza vector_cosine_ops para calcular similaridade de ângulo
CREATE INDEX idx_document_chunks_hnsw_cosine
ON document_chunks USING hnsw (embedding
vector_cosine_ops)
WITH (m \= 16, ef_construction \= 128); --

-- Indexação lexical GIN na coluna gerada para otimizar
pesquisas baseadas em palavras-chave [cite: 60]
CREATE INDEX idx_document_chunks_gin_lexical ON
document_chunks USING GIN (content_tsv);
```

Integração do Backend de Aplicação (Python + Psycpg 3)

O script abaixo demonstra de forma clara como estruturar o processamento no backend da aplicação utilizando a biblioteca psycpg para interagir diretamente com o PostgreSQL⁶¹. O backend é encarregado de extrair textos, obter os *embeddings* de alta dimensionalidade via chamadas locais de NLP e acionar as queries de busca híbrida com busca iterativa ativa para alimentar a inferência do LLM¹¹.

```
Python
import os
import numpy as np
import psycpg
from pgvector.psycpg import register_vector # [cite: 63, 64]
from sentence_transformers import SentenceTransformer #
[cite: 61, 64]

class RetrievalPipeline:
    def __init__(self, db_connection_string: str):
        self.db_dsn \= db_connection_string
        # Inicializa o modelo local de NLP (768 dimensões expandidas
        para 1536 via Matryoshka se aplicável,
        # ou modelo local para gerar embeddings compatíveis de forma
        determinística)
```

```
self.model \= SentenceTransformer("all-MiniLM-L6-v2") # [cite: 61]
```

```
def \_get\_embedding(self, text: str) \-> np.ndarray:
    \# Codifica o texto em formato float32 e normaliza para otimizar o cosseno
    embedding \= self.model.encode(text, normalize\_embeddings=True) \# \[cite:
    return np.array(embedding, dtype=np.float32) \# \[cite: 65\]

def execute\_hybrid\_search(self, tenant\_id: str, query\_text: str, limit: int):
    \# Gera o vetor representativo do prompt de consulta do usuário
    query\_vector \= self.\_get\_embedding(query\_text)

    results \= \[\]

    \# Estabelece conexão direta e transacional com o PostgreSQL
    with psycopg.connect(self.db\_dsn) as conn:
        \# Registra o suporte ao tipo de dados vector do pgvector no driver psycopg
        register\_vector(conn) \# \[cite: 63, 64\]

        with conn.cursor() as cur:
            \# 1\.. Configura as variáveis de sessão para otimização da busca vetorial
            cur.execute("SET LOCAL hnsw.ef\_search \= 100;") \# \[cite: 6, 38\]
            \# Ativa o escaneamento iterativo em modo de ordenação relaxada para melhorar a performance
            cur.execute("SET LOCAL hnsw.iterative\_scan \= 'relaxed\_order';")

            \# Executa a query híbrida unificando os filtros de metadados relacionais e de similaridade
            \# O operador \<=> representa a distância de cosseno \[cite: 6, 8\]
            hybrid\_query \= """
                SELECT
                    c.id,
                    c.content,
                    d.title,
                    1 \- (c.embedding \<=> %s) AS similarity\_score
                FROM document\_chunks c
                JOIN documents d ON c.document\_id \= d.id
                WHERE c.tenant\_id \= %s
                       AND d.created\_at \>= '2026-01-01 00:00:00+00'
                ORDER BY c.embedding \<=> %s
```

```

LIMIT %s;
"""

cur.execute(hybrid\_query, (query\_vector, tenant\_id, query\_vector))
rows \= cur.fetchall()

for row in rows:
    results.append({
        "id": row[0],
        "content": row[1],
        "source\_title": row[2],
        "similarity": float(row[3])
    })
return results

def generate\_rag\_context(self, tenant\_id: str, query\_text: str) \-> str:
    \# Recupera os trechos mais relevantes do banco do inquilino especificado
    matched\_chunks \= self.execute\_hybrid\_search(tenant\_id, query\_text)

    \# Consolida os dados recuperados em um bloco de texto unificado
    context\_blocks \= []
    for idx, chunk in enumerate(matched\_chunks):
        block \= f"--- Documento Fonte: {chunk['source\_title']} (Similaridad
        block \+= f"{chunk['content']}\n"
        context\_blocks.append(block)

    full\_context \= "\n".join(context\_blocks)

    \# Constrói o prompt final blindado para alimentação semântica do LLM downs
    prompt\_template \= (
        "Você é um assistente de IA focado em responder dúvidas corporativas co
        "Utilize os fragmentos de contexto abaixo para elaborar sua resposta.\n
        "Responda única e estritamente com base nas informações providas.\n"
        "Caso a resposta não possa ser formulada a partir do contexto, informe
        f"=== CONTEXTO DISPONÍVEL \===\n{full\_context}\n"
        f"=== PERGUNTA DO USUÁRIO \===\n{query\_text}\n\n"
        "Resposta Final:"

```



```
)  
return prompt\_template
```

Compilação de Benchmarks e Casos de Estudo

Dados Empíricos de Desempenho

Os benchmarks compilados nesta seção refletem o comportamento observado sob cargas sintéticas de *embeddings* de alta dimensionalidade (1000 dimensões padrão float32, compatíveis com a infraestrutura das APIs clássicas de IA) executados sob condições otimizadas de hardware de nuvem em instâncias com SSD NVMe local e barramento dedicado de memória.

A tabela a seguir apresenta métricas de latência mediana de busca (p50), latência p99 para verificação de cauda de concorrência, consultas por segundo (QPS) processadas e a taxa média de revocação (recall) comparando:

- 1. Busca Sequencial (KNN Exato):** Sem uso de índices vetoriais.
- 2. Índice HNSW:** Ajustado para alta revocação ($m = 16$, $ef_construction = 128$ na compilação, e sintonia dinâmica de busca variando entre $ef_search = 40$ e $ef_search = 100$).
- 3. Índice IVFFlat:** Variando as configurações de listas de centroides de Voronoi e número de partições vasculhadas ativamente no momento da query (probes).

Tamanho da Tabela (Vetores)	Algoritmo de Indexação	Parâmetros de Execução	Latência p50 (ms)	Latência p99 (ms)	Taxa de Recall (%)	Throughput (QPS)
10.000 (10k)		Nenhum (Exaustivo)	4.80	7.90	100.0%	208

Tamanho da Tabela (Vetores)	Algoritmo de Indexação	Parâmetros de Execução	Latência p50 (ms)	Latência p99 (ms)	Taxa de Recall (%)	Throughput (QPS)
	Seq Scan (KNN Exato)					
	HNSW35	ef_search \= 40 [cite: 67]	0.85	1.80	99.8%	1170
	IVFFlat39	lists=100, probes=10 [cite: 39]	1.20	2.50	96.5%	830
100.000 (100k)	Seq Scan (KNN Exato)	Nenhum (Exaustivo)35	48.00	72.00	100.0%50	21
	HNSW23	ef_search \= 40 [cite: 23, 67]	3.00	9.2067	98.5%23	32567
	HNSW67	ef_search \= 100 [cite: 6]	4.10	11.50	99.7%	240
	IVFFlat39	lists=316, probes=31 [cite: 39]	5.50	12.80	94.0%	180
1.000.000 (1M)	Seq Scan (KNN Exato)	Nenhum (Exaustivo)35	650.0035	890.0068	100.0%50	1.568
	HNSW23	ef_search \= 40 [cite: 23]	7.0023	22.0023	92.0%23	142
	HNSW23		15.0023	45.0023	98.0%23	66

Tamanho da Tabela (Vetores)	Algoritmo de Indexação	Parâmetros de Execução	Latência p50 (ms)	Latência p99 (ms)	Taxa de Recall (%)	Through (QPS)
		ef_search \= 100 [cite: 23, 52]				
	IVFFlat23	lists=1000, probes=100 [cite: 39]	45.00	95.00	91.5%	22

Análise de Desempenho e Comportamento de Escala

A análise empírica revela que a busca sequencial exata exibe um comportamento estritamente linear no tempo de execução, escalando de 10ms na escala de 10k para proibitivos 100ms quando a base alcança 1 milhão de vetores³⁵. Esse tempo inviabiliza qualquer uso em aplicações em tempo real, onde as restrições de tempo de resposta exigem latências de consulta inferiores a 50 ms⁵¹. Consequentemente, a busca sem índices torna-se impraticável para volumes de dados acima de 50.000 registros, justificando o uso de algoritmos de aproximação¹³.

Ao manter o volume de dados em 100k, o HNSW com ef_search \= 40 opera com latência p50 excepcional de apenas 45ms, com taxas de revocação de 91.5%²³. Entretanto, à medida que a base cresce para 1 milhão de vetores, a manutenção de uma alta taxa de revocação (91.5%) com o parâmetro ef_search \= 100 eleva a latência mediana para 45ms [image62] ms, apresentando uma cauda de latência p99 que atinge 95ms²³.

Sob condições de alta concorrência concorrente de consultas, esses p99 degradam rapidamente se os índices começarem a competir com operações de gravação e atualizações de dados que causam descarte de páginas no buffer cache do

PostgreSQL²⁸. Esse descarte força o banco a executar operações aleatórias de leitura física em disco para percorrer a estrutura do grafo, degradando a vazão total do sistema (QPS)²⁸.

O índice IVFFlat exibe sua vantagem histórica de baixo custo na compilação estrutural e retenção física compacta em disco e memória RAM, que se alinha muito próximo a  o tamanho dos vetores puros³⁵. No entanto, em termos de eficiência de busca por latência, ele demonstra as desvantagens de sua topologia simplificada na escala de 1 milhão de vetores³². Para manter a latência de execução minimamente aceitável de  ms, o algoritmo restringe o número de partições exploradas (probes ≈ 100), o que resulta em uma queda na revocação para ²³. Essa perda de precisão semântica significa que quase uma em cada dez respostas mais relevantes será perdida pelo indexador, prejudicando o contexto de grounded do LLM na geração de respostas³⁵.

A Transição Arquitetural para Escalas Avançadas (Mais de 10M de Vetores)

Conclui-se que o HNSW de base puramente em RAM apresenta excelente desempenho para bases de dados contendo até 5 a 10 milhões de vetores, desde que o servidor possua capacidade financeira para acomodar o tamanho do índice totalmente residente nos buffers do SGBD²³.

Quando o volume ultrapassa o patamar de dezenas de milhões de registros, o custo de hardware de memória RAM dedicada para reter o grafo HNSW torna-se proibitivo⁵². Sob esta restrição de recursos, a engenharia de sistemas de bancos de dados propõe o uso do ecossistema complementar pgvector^{scale12}.

Essa extensão introduz o algoritmo **StreamingDiskANN**, desenvolvido originalmente sob pesquisas da Microsoft Corporation¹². O DiskANN substitui o HNSW por uma estrutura de grafo do tipo Vamana, desenhada especificamente para reter o corpo denso do índice em unidades físicas de estado sólido de alta performance (SSD NVMe) enquanto preserva apenas um índice esparsa e compressões de vetores na memória RAM

através de técnicas de Quantização Binária Estatística (SBQ)⁶⁹. Isso viabiliza buscas com latência de milissegundos sobre centenas de milhões de vetores, minimizando a pegada financeira da infraestrutura de IA⁵².

Referências Bibliográficas

- KANE, Andrew. *pgvector*: Open-source vector similarity search for Postgres. GitHub Repository, 2026. Disponível em: <https://github.com/pgvector/pgvector>.⁶
- MALKOV, Yu. A.; YASHUNIN, D. A. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, v. 42, n. 4, p. 824-836, 2018.⁴⁵
- PUGH, William. Skip Lists: A Probabilistic Alternative to Balanced Trees. *Communications of the ACM*, v. 33, n. 6, p. 668-676, 1990.⁴²
- SUPABASE. *HNSW Indexes in pgvector*. Supabase Engineering Blog, 2026. Disponível em: <https://supabase.com/docs/guides/ai/vector-indexes/hnsw-indexes>.³¹
- TIMESCALE. *pgvector scale: Scaling vector search to 50M+ on PostgreSQL*. Timescale Technical Engineering Post, 2025. Disponível em: <https://github.com/timescale/pgvector scale>.⁷⁰

Referências citadas

1. PGVector: Transforming PostgreSQL into a Powerful Vector Database for AI Applications | by Amit Dhiman | Medium, <https://medium.com/@amitdhiman91/pgvector-transforming-postgresql-into-a-powerful-vector-database-for-ai-applications-d2716689b50d>
2. NeuronDB Vector vs pgvector: Technical Comparison - DEV Community, https://dev.to/neurondb_support_d73fa7ba/neurondb-vector-vs-pgvector-technical-comparison-4mmh

3. Speed up PostgreSQL® pgvector queries with indexes, <https://aiven.io/developer/postgresql-pgvector-indexes>
4. Turning PostgreSQL Into a Vector Database With pgvector - DEV Community, <https://dev.to/tigerdata/postgresql-extensions-turning-postgresql-into-a-vector-database-with-pgvector-38gd>
5. Faster similarity search performance with pgvector indexes | Google Cloud Blog, <https://cloud.google.com/blog/products/databases/faster-similarity-search-performance-with-pgvector-indexes>
6. pgvector/pgvector: Open-source vector similarity search for Postgres - GitHub, <https://github.com/pgvector/pgvector>
7. pgvector Distance Functions: Cosine vs L2 vs Inner Product - myDBA.dev, <https://mydba.dev/blog/pgvector-distance-functions>
8. pgvector similarity search: Basics, tutorial and best practices, <https://www.instaclustr.com/education/vector-database/pgvector-similarity-search-basics-tutorial-and-best-practices/>
9. PostgreSQL® Vector Search with pgvector - Aiven, <https://aiven.io/blog/aiven-for-postgres-supports-pgvector>
10. The Impacts of Data, Ordering, and Intrinsic Dimensionality on Recall in Hierarchical Navigable Small Worlds - arXiv, <https://arxiv.org/html/2405.17813v1>
11. RAG with Postgres pgvector in 2026: the full TypeScript pipeline. - DEV Community, <https://dev.to/thegdsks/rag-with-postgres-pgvector-in-2026-the-full-typescript-pipeline-2lbd>
12. PostgreSQL 18 + pgvector: The Definitive Guide to Building Production-Grade RAG Pipelines | by Mohit soni | Medium, <https://medium.com/@mohitsoni/postgresql-18-pgvector-the-definitive-guide-to-building-production-grade-rag-pipelines-239ee9c0e56f>
13. pgvector with Node.js: Production Vector Search Without a Second Database - Library, <https://www.grizzypeaksoftware.com/library/pgvector-vector-search-in-postgresql-j96fbhd9>

14. PostgreSQL - Grokipedia, <https://grokipedia.com/page/PostgreSQL>
15. What Is pgvector? PostgreSQL Vector Search Extension Explained - VeloDB, <https://www.velodb.io/glossary/what-is-pgvector>
16. Vector Databases 2026: Pinecone vs Weaviate vs pgvector - Tech Bytes, <https://techbytes.app/posts/vector-databases-2026-pinecone-vs-weaviate-vs-pgvector/>
17. pgvector vs Qdrant: 5 key differences and how to choose - NetApp Instaclustr, <https://www.instaclustr.com/education/vector-database/pgvector-vs-qdrant-5-key-differences-and-how-to-choose/>
18. PostgreSQL: Advanced Open-Source Insights | PDF | Postgre Sql | Database Index - Scribd, <https://www.scribd.com/document/909749337/PostgreSQL-Architecture-Deep-Dive-Brijesh-Mehra>
19. Vector Databases for .NET Compared: Azure AI Search, Qdrant, Cosmos DB & pgvector, <https://medium.com/@bhargavkoya56/vector-databases-for-net-compared-azure-ai-search-qdrant-cosmos-db-pgvector-3f4a0aae5d19>
20. Unified Graph-RAG in a Single Postgres Engine | by Data Do GmbH | May, 2026 | Medium, <https://medium.com/@DataDo/unified-graph-rag-in-a-single-postgres-engine-001a7f815589>
21. Debunking 6 common pgvector myths - DEV Community, <https://dev.to/gwenshap/debunking-6-common-pgvector-myths-knh>
22. How to deal with different vector-dimensions for embeddings and search with pgvector? - Stack Overflow, <https://stackoverflow.com/questions/77883843/how-to-deal-with-different-vector-dimensions-for-embeddings-and-search-with-pgve>
23. pgvector vs Pinecone: When PostgreSQL Is Enough for Vector Search - Boolean & Beyond, <https://www.booleanbeyond.com/en/insights/pgvector-vs-pinecone-when-postgresql-enough-vector-search>

24. Load vector embeddings up to 67x faster with pgvector and Amazon Aurora - AWS, <https://aws.amazon.com/blogs/database/load-vector-embeddings-up-to-67x-faster-with-pgvector-and-amazon-aurora/>
25. Best practices for using pgvector, https://postgresql.us/events/pgconfnyc2024/sessions/session/1862/slides/172/pgvector_best_practices_pgconfnyc2024.pdf
26. IVFFLAT QPS too low · Issue #661 - GitHub, <https://github.com/pgvector/pgvector/issues/661>
27. Storing and querying vector data in Postgres with pgvector - pganalyze, <https://pganalyze.com/blog/5mins-postgres-vectors-pgvector>
28. pgvector Limitations - ParadeDB, <https://www.paradedb.com/learn/postgresql/pgvector-limitations>
29. PostgreSQL + pgvector for Relational Data and AI Embeddings Guide - Mad Devs, <https://maddevs.io/writeups/pgvector-postgresql-relational-data-ai-embeddings/>
30. pgvector, a guide for DBA - Part 2: Indexes (update march 2026) - dbi services, <https://www.dbi-services.com/blog/pgvector-a-guide-for-dba-part-2-indexes-update-march-2026/>
31. HNSW indexes | Supabase Docs, <https://supabase.com/docs/guides/ai/vector-indexes/hnsw-indexes>
32. pgvector Review 2026: Vector Search Inside PostgreSQL - PE Collective, <https://pecollective.com/tools/pgvector/>
33. pgvector performance: Benchmark results and 5 ways to boost performance, <https://www.instaclustr.com/education/vector-database/pgvector-performance-benchmark-results-and-5-ways-to-boost-performance/>
34. Supercharging vector search performance and relevance with pgvector 0.8.0 on Amazon Aurora PostgreSQL | AWS Database Blog, <https://aws.amazon.com/blogs/database/supercharging-vector-search-performance-and-relevance-with-pgvector-0-8-0-on-amazon-aurora-postgresql/>

35. HNSW vs IVFFlat: How to Choose the Right Vector Index - BigData Boutique, <https://bigdataboutique.com/blog/hnsw-vs-ivfflat-how-to-choose-the-right-vector-index>
36. PGVector: HNSW vs IVFFlat — A Comprehensive Study | by BavalpreetSinghh - Medium, <https://medium.com/@bavalpreetsinghh/pgvector-hnsw-vs-ivfflat-a-comprehensive-study-21ce0aaab931>
37. Enable AI-Ready Vector Search with PGVector - PolarDB - Alibaba Cloud, <https://www.alibabacloud.com/help/en/polardb/polardb-for-oracle/pgvector-for-oracle>
38. How to optimize performance when using pgvector - Azure Cosmos DB for PostgreSQL, <https://learn.microsoft.com/en-us/azure/cosmos-db/postgresql/howto-optimize-performance-pgvector>
39. ApsaraDB RDS: Use the pgvector extension to test performance based on IVF indexes - Alibaba Cloud, <https://www.alibabacloud.com/help/doc-detail/2869991.html>
40. pgvector Index Selection: IVFFlat vs HNSW for PostgreSQL Vector Search - Medium, <https://medium.com/@philmcc/pgvector-index-selection-ivfflat-vs-hnsw-for-postgresql-vector-search-6eff26aaa90c>
41. Understanding the Real Challenges of HNSW Nearest Neighbor Search by Re-implementing from the Paper: Deep Dive into pgvector - Zenn, <https://zenn.dev/jobmore/articles/hnsw-pgvector-deep-dive?locale=en>
42. Hierarchical Navigable Small Worlds (HNSW) - Pinecone, <https://www.pinecone.io/learn/series/faiss/hnsw/>
43. MySQL Day 48: PGVector HNSW index - Medium, <https://medium.com/sys-base/mysql-day-48-pgvector-hnsw-index-ab3560a94a04>
44. Running pgvector in production on Amazon Aurora PostgreSQL | AWS Database Blog, <https://aws.amazon.com/blogs/database/running-pgvector-in-production-on-amazon-aurora-postgresql/>
45. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs - Khoury College of Computer Sciences, <https://>

khoury.northeastern.edu/home/pandey/courses/cs7270/fall25/papers/vectordb/HNSW.pdf

46. A cluster-based iterative search approach over HNSW, <https://sol.sbc.org.br/index.php/eramians/article/download/39410/39182/>
47. Benchmarking pgvector RAG performance across different dataset sizes - Mastra, <https://mastra.ai/blog/pgvector-perf>
48. How to calculate amount of RAM required for serving X N-dimensional vectors with pgvector (HNSW)? - Stack Overflow, <https://stackoverflow.com/questions/77401874/how-to-calculate-amount-of-ram-required-for-serving-x-n-dimensional-vectors-with>
49. Pgvector vs. Qdrant: Open-Source Vector Database Comparison - Tiger Data, <https://www.tigerdata.com/blog/pgvector-vs-qdrant>
50. The AI Engineer's Playbook: Mastering Vector Search & Management (Part 2) - Medium, <https://medium.com/data-science-collective/the-ai-engineers-playbook-mastering-vector-search-management-part-2-7a74b8038bc5>
51. Best Vector Databases in 2026: A Complete Comparison Guide - Firecrawl, <https://www.firecrawl.dev/blog/best-vector-databases>
52. Optimizing Vector Search at Scale: Lessons from pgvector & Supabase Performance Tuning | by Dikhyant Krishna Dalai | Medium, <https://medium.com/@dikhyantkrishnadalai/optimizing-vector-search-at-scale-lessons-from-pgvector-supabase-performance-tuning-ce4ada4ba2ed>
53. Qdrant Cloud Pricing 2026: Tiers, Costs And Self-Hosted Crossover - RankSquire, <https://ranksquire.com/2026/04/19/qdrant-cloud-pricing-2026/>
54. pgvector - Tecnisys Docs, <https://docs.tecnisys.com.br/pgsys-ecosystem/pdf/pgvector/0.8.0/pgvector-0.8.0.pdf>
55. <https://ftp.sun.ac.za/ftp/pub/mirrors/apt.postgresql.org/dists/bookworm-pgdg-testing/main/binary-s390x/Packages>

56. Build a RAG System with pgvector on Managed PostgreSQL (2026) | DanubeData, <https://danubedata.ro/blog/pgvector-rag-managed-postgres-2026>
57. Postgres + pgvector: Production Retrieval on a Budget | by Velorum - Medium, <https://medium.com/@1nick1patel1/postgres-pgvector-production-retrieval-on-a-budget-814df87df5c9>
58. A Coding Guide to Implement a pgvector-Powered Semantic, Hybrid, Sparse, and Quantized Vector Search System - MarkTechPost, <https://www.marktechpost.com/2026/05/28/a-coding-guide-to-implement-a-pgvector-powered-semantic-hybrid-sparse-and-quantized-vector-search-system/>
59. Enable pgvector and Load Embeddings | DigitalOcean Documentation, <https://docs.digitalocean.com/products/vector-databases/postgresql/how-to/load-embeddings/>
60. Understanding Semantic and Hybrid Search with Python and pgvector | by Hasan Sajedi, <https://hasansajedi.medium.com/understanding-semantic-and-hybrid-search-with-python-and-pgvector-0967e83803e6>
61. pgvector Benchmarks, Per Tier: Honest, Reproducible Numbers from Solo to Scale, <https://rivestack.io/blog/pgvector-performance-nvme-vs-cloud-ssd-benchmarks>
62. Is Actian VectorAI DB the Best Embedded pgvector Alternative?, <https://www.actian.com/blog/developer/is-actian-vectorai-db-the-best-embedded-pgvector-alternative/>
63. pgvector vs pgvector scale: When Vanilla Isn't Enough - Rivestack, <https://rivestack.io/blog/pgvector-vs-pgvector scale>
64. Advanced Vector Workloads with pgvector scale and Hybrid Search - DigitalOcean Docs, <https://docs.digitalocean.com/products/vector-databases/postgresql/how-to/advanced-workloads/>

Apêndice A — Código SQL

Schema (01_schema.sql)

```
-- =====
-- Schema para Demo de pgvector - PDB 2026.1
-- =====

CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Tabela de documentos mestre
CREATE TABLE documents (
    id BIGSERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    author TEXT,
    created_at TIMESTAMPTZ DEFAULT now(),
    tenant_id UUID NOT NULL
);

-- Tabela de fragmentos (chunks) com embedding
CREATE TABLE document_chunks (
    id BIGSERIAL PRIMARY KEY,
    document_id BIGINT NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    chunk_index INT NOT NULL,
    content TEXT NOT NULL,
    embedding vector(384),
    tenant_id UUID NOT NULL,
    metadata JSONB,
    content_tsv tsvector
    GENERATED ALWAYS AS (to_tsvector('portuguese', content)) STORED,
    UNIQUE (document_id, chunk_index)
);

-- Indices
CREATE INDEX idx_chunks_tenant ON document_chunks (tenant_id);
```

```
CREATE INDEX idx_chunks_document ON document_chunks (document_id);

CREATE INDEX idx_chunks_hnsw_cosine
    ON document_chunks USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 128);

CREATE INDEX idx_chunks_gin_lexical
    ON document_chunks USING GIN (content_tsv);
```

Queries de Exemplo (02_queries.sql)

```
-- =====
-- Queries de Exemplo - pgvector
-- =====

-- 1. KNN exato (sem indice) - busca linear
-- Lento em grandes volumes, mas recall 100%
SELECT id, content, 1 - (embedding <=> '[0.1,0.2,...]') AS similarity
FROM document_chunks
WHERE tenant_id = '...'
ORDER BY embedding <=> '[0.1,0.2,...]'
LIMIT 5;

-- 2. ANN com HNSW (indice ativado automaticamente)
-- Configura parametros de consulta
SET LOCAL hnsw.ef_search = 100;
SET LOCAL hnsw.iterative_scan = 'relaxed_order';

SELECT id, content,
       1 - (embedding <=> '[0.1,0.2,...]') AS similarity
FROM document_chunks
WHERE tenant_id = '...'
ORDER BY embedding <=> '[0.1,0.2,...]'
LIMIT 5;

-- 3. Busca hibrida: vetorial + full-text
```

```

SET LOCAL hnsw.ef_search = 100;

WITH semantic AS (
    SELECT id, content,
           1 - (embedding <=> '[0.1,0.2,...]') AS score
    FROM document_chunks
    WHERE tenant_id = '...'
    ORDER BY embedding <=> '[0.1,0.2,...]'
    LIMIT 20
),
lexical AS (
    SELECT id, content,
           ts_rank(content_tsv, plainto_tsquery('portuguese', 'busca texto')) AS score
    FROM document_chunks
    WHERE tenant_id = '...'
    AND content_tsv @@ plainto_tsquery('portuguese', 'busca texto')
    LIMIT 20
)
SELECT id, content, 'semantic' AS match_type, score
FROM semantic
UNION ALL
SELECT id, content, 'lexical' AS match_type, score
FROM lexical
ORDER BY score DESC
LIMIT 5;

-- 4. Busca exata para comparar recall vs HNSW
SELECT id, content
FROM document_chunks
WHERE tenant_id = '...'
ORDER BY embedding <=> '[0.1,0.2,...]'
LIMIT 5;

```

Apêndice B — Scripts Python

Os scripts Python estão disponíveis em `demo/src/` no repositório: - `generate_embeddings.py` — Gera embeddings com sentence-transformers e insere no PostgreSQL - `hybrid_search.py` — Busca híbrida (vetorial + full-text) com deduplicação - `rag_pipeline.py` — Pipeline RAG completo (recuperação + geração de resposta)

Apêndice C — Como Reproduzir

```
cd demo
docker compose up -d
pip install -r src/requirements.txt
python src/generate_embeddings.py
python src/hybrid_search.py
python src/rag_pipeline.py
```